School of CSE, REVA University

Codebase-Wide Version Migration & Refactoring Agent

Task 2: Team Overview, Module Description and Technology Stack

1. Project Details

Item	Details
Project Title	Codebase-Wide Version Migration & Refactoring Agent
Problem Statement Identifier	CAIML_Nimrita_Koul_3_PS_209
Faculty Guide	Dr. Nimrita Koul
Center	Center for Artificial Intelligence & Machine Learning
School and University	School of CSE, REVA University

2. Team Overview

The project is carried out by a team of five students. The team members are listed below.

S. No.	Name	SRN
1	Adithya Naik	R23EF014
2	Prajwal Ragavendra Hawaldar	R23EF183
3	Verril Vaz	R23EN179
4	Nandan P	R23EH082
5	Prabanjan R	R23EF181

3. Project Overview

The project develops an autonomous agent that upgrades a multi-module Python codebase to accommodate breaking changes in a language or library version. Examples are the migration of database code from SQLAlchemy 1.4 to 2.0, the conversion of a Python 2 codebase to Python 3, and the upgrade of Python 3.8 code to Python 3.12. The primary migration for development and evaluation is SQLAlchemy 1.4 to 2.0.

Given a migration specification and a target repository, the agent first maps all affected call sites using static analysis, then plans an edit order over the dependency graph, applies the edits in batches, and runs the linter and test suite after each batch. Failures are traced back to the change that caused them and corrected until the test suite passes. A graph-based memory records the state of every file so that the agent can work over a long execution within a fixed token budget. The outputs are a unified patch file, structured JSON logs of the agent's decisions, and benchmark metrics.

The system is divided into nine modules, described in Section 4. Modules 1 to 3 prepare the migration, modules 4 to 7 form the execution loop, and modules 8 and 9 produce and evaluate the results.

4. Module Description

4.1 Migration Specification and Configuration Module

This module receives the migration to be performed and prepares it for the rest of the system. A specification names the deprecated APIs or syntax, describes their replacements, and can include deterministic transformation rules. The module also reads the run configuration, which contains the repository path, the token, step and time limits, the retry limit, and a dry-run switch. New migrations are added as specification files, so the core engine does not need to change.

Input: Specification file, repository path and run configuration.

Output: A validated specification object and configuration passed to all other modules.

Techniques: Declarative rule files, schema validation, command-line interface.

4.2 Static Analysis and Call-Graph Mapping Module

This module analyses the repository before any edit is made. It parses every source file into an abstract syntax tree, builds the import graph and a symbol table that resolves aliases and re-exports, and builds a call graph. It then lists every call site of the deprecated APIs with file, line and enclosing function, and computes the impact set, which contains the transitive callers whose contract may change. Sites that cannot be resolved statically, for example those using dynamic attribute access, are reported separately. Files that fail to parse are recorded without stopping the analysis. The module also contains an independent oracle detector that counts deprecated usages, which is used to measure migration completeness.

Input: Repository source code and the migration specification.

Output: Import graph, call graph, list of breaking call sites, impact set and list of unresolved sites.

Techniques: Python ast module, LibCST, tree-sitter, symbol resolution with Jedi or Pyright, NetworkX graphs.

4.3 Planning and Batching Engine

This module decides the order in which files are changed. It orders files topologically over the dependency graph so that a module is changed after the modules it depends on. Cyclic dependencies are collapsed into strongly connected components and handled as a single unit, which prevents a repair in one module from undoing a change in another. Files are then grouped into batches of a size that can be verified together. The plan is updated when a failure shows that a further file needs to change.

Input: Dependency graph, impact set and configuration limits.

Output: An ordered list of batches, and updates to the plan during execution.

Techniques: Topological sorting, Tarjan's strongly connected components algorithm, batch sizing rules.

4.4 Graph-Based Memory and Context Manager

This module stores the working state of the migration outside the language model prompt. Every file is a node in the dependency graph with a state, which is one of pending, analysed, edited, verified or failed. The module also stores pending dependency fixes and short summaries of completed work. Before each model call it assembles a prompt from the current file, its neighbouring nodes, the open failures and the relevant summaries, and keeps the prompt within the configured token budget. The state is saved to disk so that a run can be resumed.

Input: Graph, plan, edit results and failure reports from the other modules.

Output: Current file states, pending fixes and a token-bounded context for each model call.

Techniques: Graph state store (JSON or SQLite), summarisation of completed nodes, token counting and budgeting.

4.5 Edit Engine

This module changes the source code. Mechanical transformations are applied first by deterministic codemods, which give repeatable results and preserve comments and formatting. Sites that need judgement are passed to the language model, which returns a patch. Each patch is checked before it is accepted: it must apply cleanly and the resulting file must still parse. A patch that fails these checks is rejected and the model is asked again with the reason.

Input: Current batch, call-site information and the context prepared by module 4.

Output: Edited files and the corresponding diffs.

Techniques: LibCST codemods, LLM-generated unified diffs, patch validation, Git.

4.6 Sandboxed Verification Module

This module runs the linter and the tests after each batch and reports the results. Execution takes place inside a Docker container with limits on resources and network access, and the original repository is never modified in place. The affected tests are run after each batch and the full test suite is run at the end. A baseline run before any edit records which tests already passed, so that regressions can be told apart from earlier failures. Each batch is checkpointed with Git so that a failed batch can be rolled back.

Input: Edited repository state and test selection from the plan.

Output: Linter results, test results, raw stack traces and a checkpoint or rollback of the batch.

Techniques: Docker, pytest, Ruff, flake8, Git commits and reverts.

4.7 Failure Analysis and Self-Correction Module

This module handles failures reported by module 6. It parses stack traces and linter output, maps each failing frame to a node of the dependency graph, and follows the call graph upward to find the breaking contract that caused the failure. It then asks the language model for a targeted fix at that location and sends it through the edit engine. The number of retries is limited, and the module detects repeated or oscillating patches. When a batch cannot be repaired within the limit, the batch is rolled back and marked as failed in the memory.

Input: Test and linter output, stack traces, call graph and graph memory.

Output: Targeted fix requests, updated file states, and rollback decisions.

Techniques: Stack-trace parsing, call-graph traversal, bounded retry loop, patch-history comparison.

4.8 Trajectory Logging and Patch Generation Module

This module records what the agent did and produces the final deliverables. It writes structured JSON logs that contain the initial call-graph mapping, the order of file edits, the intermediate test and linter failures, and the corrective actions, together with prompts, token counts and step counts. At the end of a run it produces one unified patch file that applies the complete migration to the repository.

Input: Events from all modules and the final repository state.

Output: JSON trajectory and state audit logs, and a unified .patch file.

Techniques: Versioned JSON schema, Git diff.

4.9 Benchmark and Evaluation Harness

This module runs the agent over a set of migration tasks and computes the evaluation metrics. The metrics are migration completeness, which is the percentage of deprecated call sites updated, the post-migration test pass rate, and the token and step overhead. It also runs baselines, such as a single-shot language model and a flat-context agent, and ablations that switch off one component at a time, so that the contribution of static analysis, graph memory, verification and codemods can be measured. Tasks are built from real repositories with upgrade histories and from synthetic repositories in which deprecated patterns are introduced in a controlled way.

Input: Benchmark tasks, agent configuration and the logs from module 8.

Output: Per-task and aggregate metric tables for the benchmarking report.

Techniques: Python scripts, pandas for aggregation, matplotlib for plots.

5. Technology Stack

The technologies below are proposed for the implementation. Items marked as to be finalised will be decided after evaluation during the early implementation phase.

Category	Technology	Use in the project
Programming language	Python 3.12	Implementation of the agent and all modules
Parsing and syntax trees	ast module, LibCST, tree-sitter	Parsing source files, locating call sites, format-preserving edits
Symbol resolution	Jedi or Pyright (to be finalised)	Resolving aliases and inherited calls
Graph processing	NetworkX	Import graph, call graph, dependency ordering
State storage	JSON files or SQLite	Persistent graph memory and run state
Testing	pytest	Running affected tests and the full suite
Linting	Ruff, flake8	Detecting syntax and style errors after edits
Version control	Git	Batch checkpoints, rollback and patch generation
Sandboxing	Docker	Isolated execution of tests and tools
Language model	Hosted API or local open model (to be finalised)	Generating patches and targeted fixes
Agent control loop	Custom Python implementation	Tool calls, planning and retry logic
Logging	Structured JSON	Trajectory and state audit logs
Evaluation and analysis	pandas, matplotlib	Metric aggregation and plots for the report
Development environment	Visual Studio Code, Windows, Linux containers	Development and reproducible execution